

BCC1313
ALGORITHMS AND COMPLEXITY

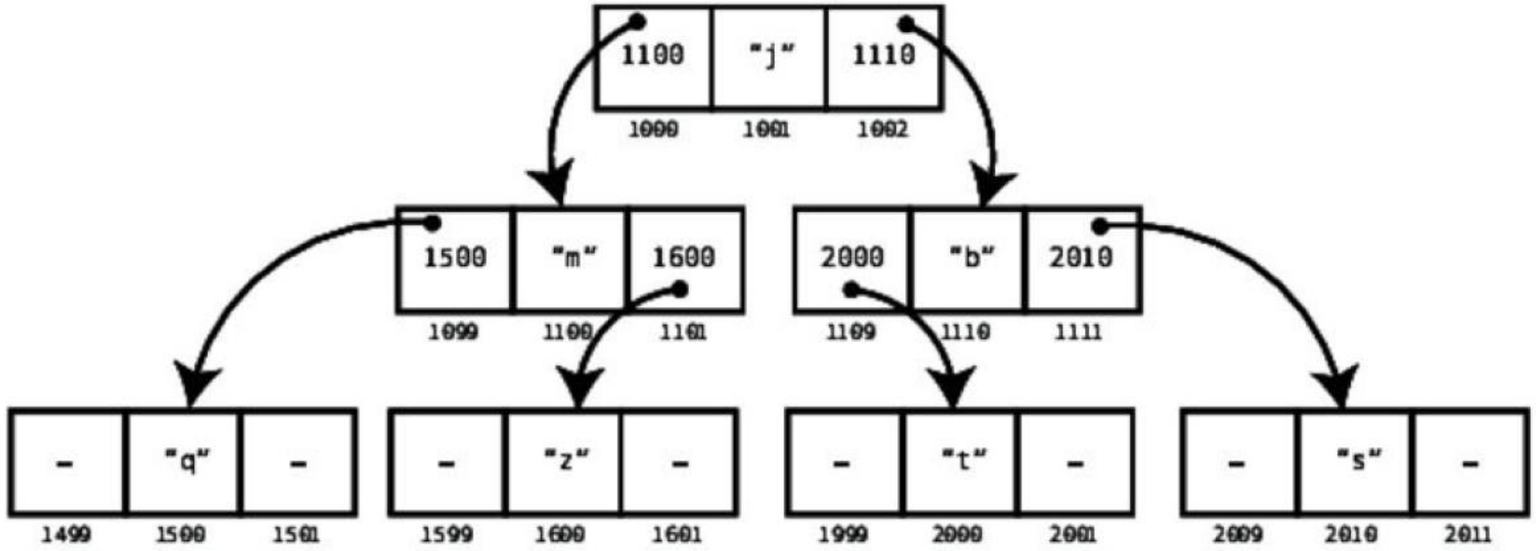
Trees

INTRODUCTION

- **Tree** is a **hierarchical model** that organizes data in a **parent-child relationship.**
- They consist of **nodes**, where each node holds a value and references to its children.

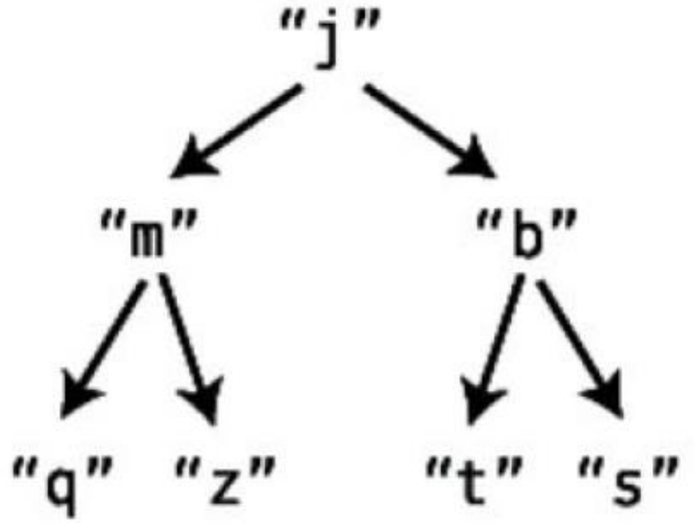
INTRODUCTION

- In a simple linked list, each node contains a link that connects this node to a single other node. A tree is also a node-based data structure, but within a tree, each node can have links to multiple nodes.



INTRODUCTION

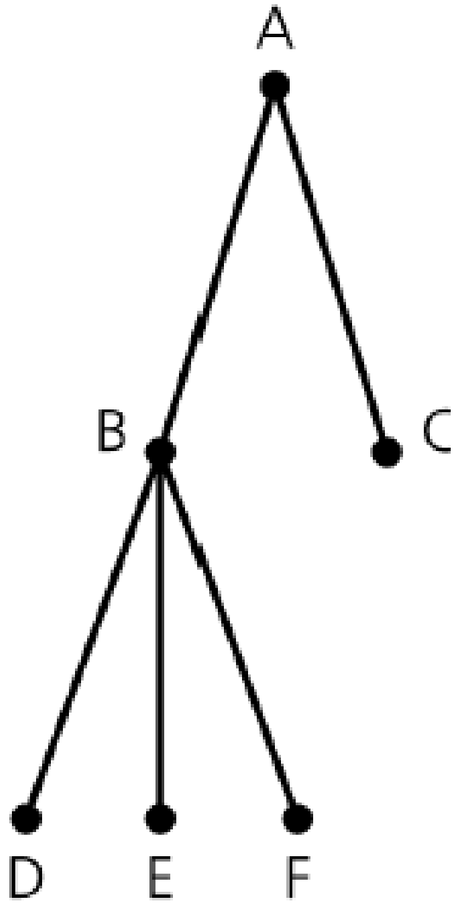
- For the sake of simplicity, we can represent this tree visually without showing all the actual links.



- Tree is a very useful in many real-world applications:
  - **Position-oriented ADT:** Binary Tree
  - **Value-oriented ADT:** Binary Search Tree.

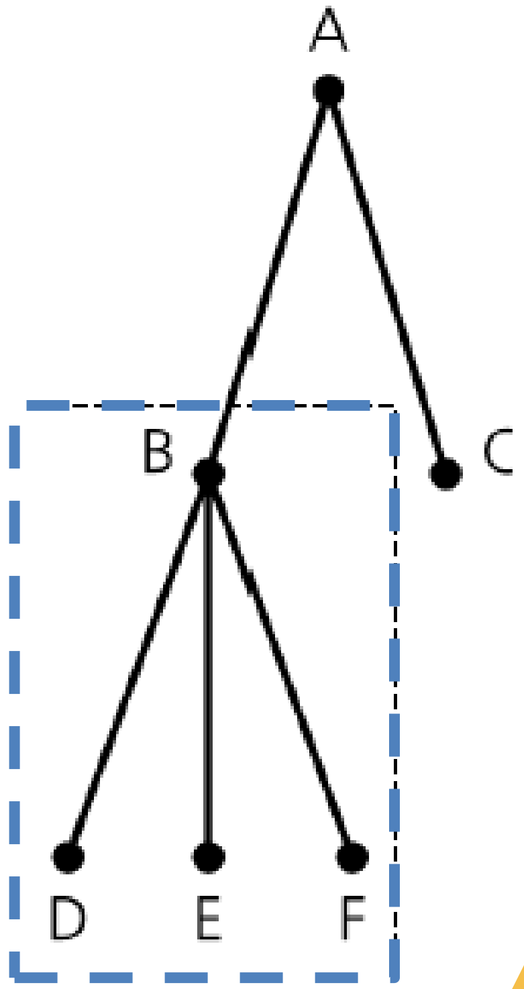
TERMINOLOGY

- Trees are composed of **nodes (vertex)** and **edges**.
- Trees are **hierarchical**
  - **Parent-child** relationship between two nodes.
  - E.g., B and C are children of A
- Each node in a tree has at most one parent
  - Except **root** has no parent. E.g. A
- A node that has no children is called a **leaf** of the tree.
  - E.g., C, D, E, F



TERMINOLOGY

- **Sibling relationship**: children of the same parent
  - E.g., B and C
- **Ancestor-descendant** relationships among nodes.
  - A is an ancestor of D
  - D is a descendant of A
  - The root of any tree is an ancestor of every node in that tree
- **Subtree** of a tree: any node and its descendants.

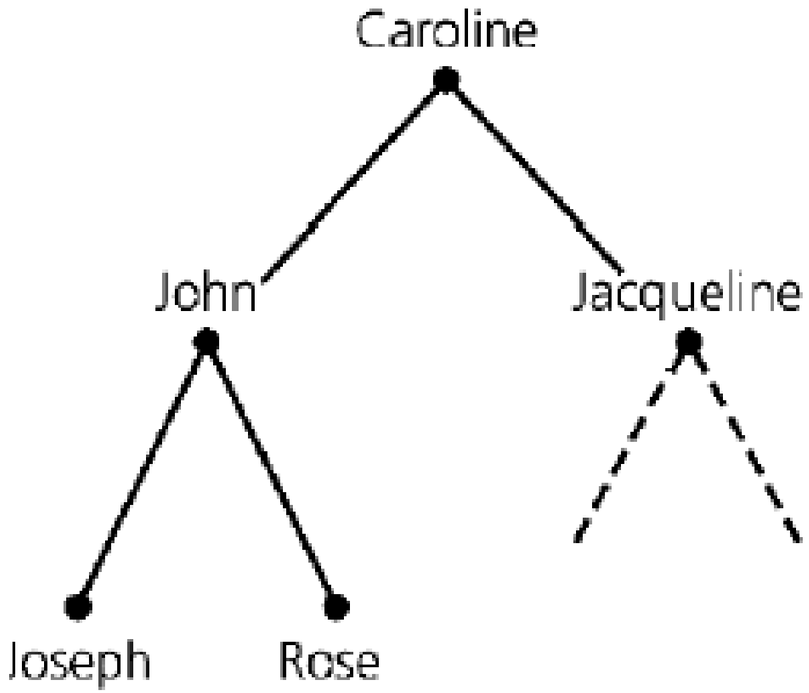


GENERAL TREES

- A **general tree, *T*** is a set of one of more nodes such that ***T*** is partitioned into disjoint subsets:
  - A single node ***r***, the root
  - Sets that are general trees, called **subtrees** of ***r***
- The hierarchical property of tree:
  - A tree can represent information that is inherently hierarchical, such as a family tree

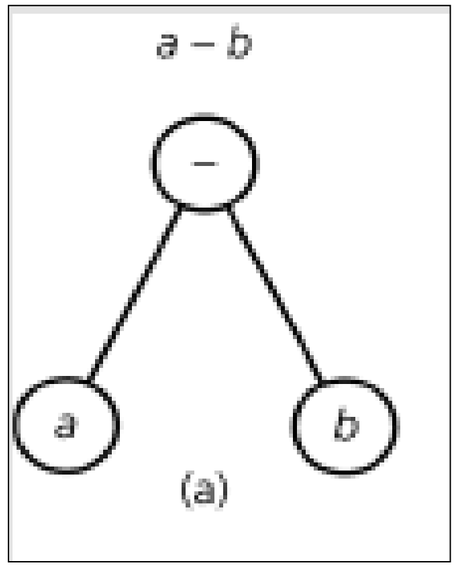
BINARY TREES

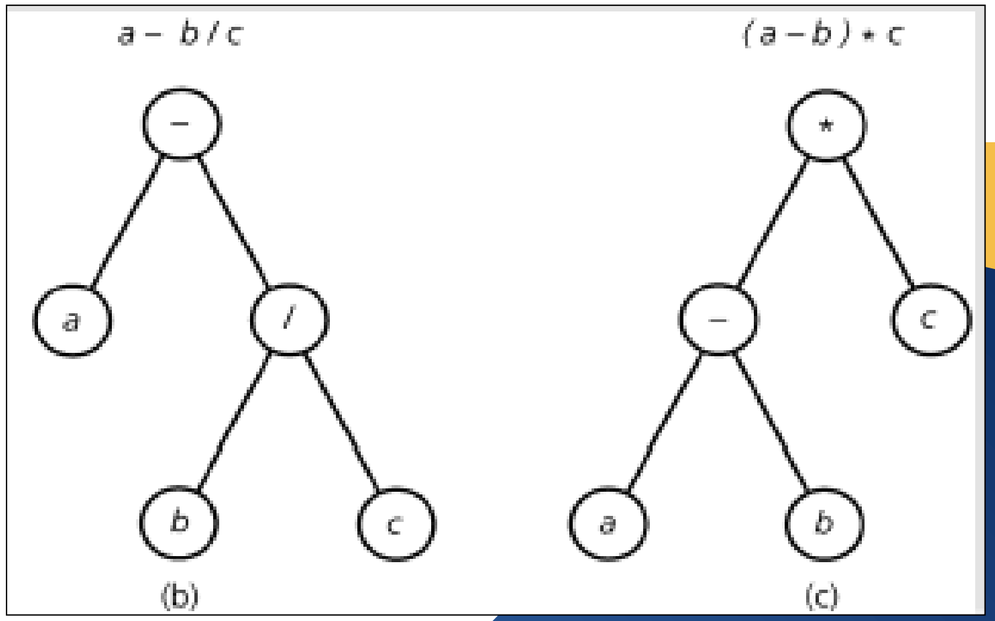
- A **binary tree** is a set ***T*** of nodes such that either:
  - ***T*** is empty, or
  - ***T*** is partitioned into three disjoint subsets:
    - A single node ***r***, the root
    - Two (possibly empty) sets that are binary trees, called the **left subtree** of ***r*** and the **right subtree** of ***r***.



BINARY TREES

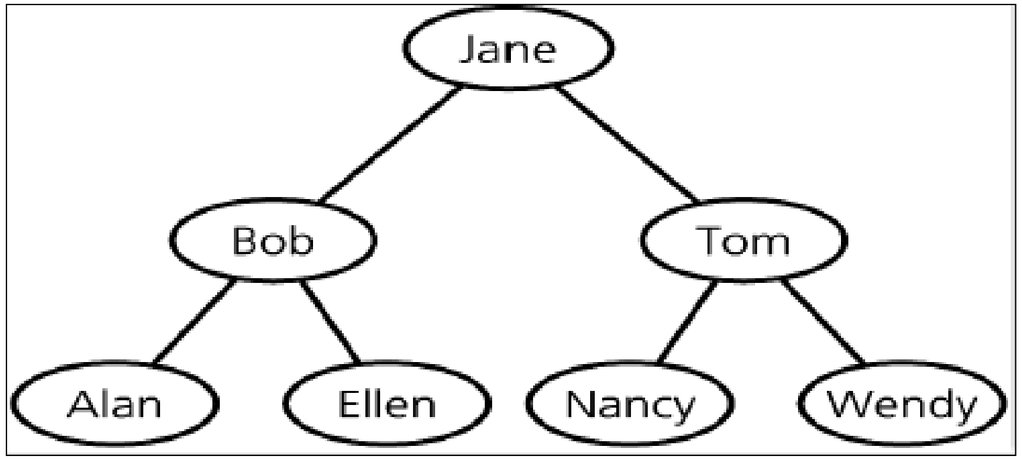
- You can use binary trees to represent **algebraic expressions**
  - **Leafs** are operands
  - Evaluation manner: **bottom-up**
    - The hierarchy specifies an unambiguous order for evaluating an expression
    - Parentheses do not appear in these tree





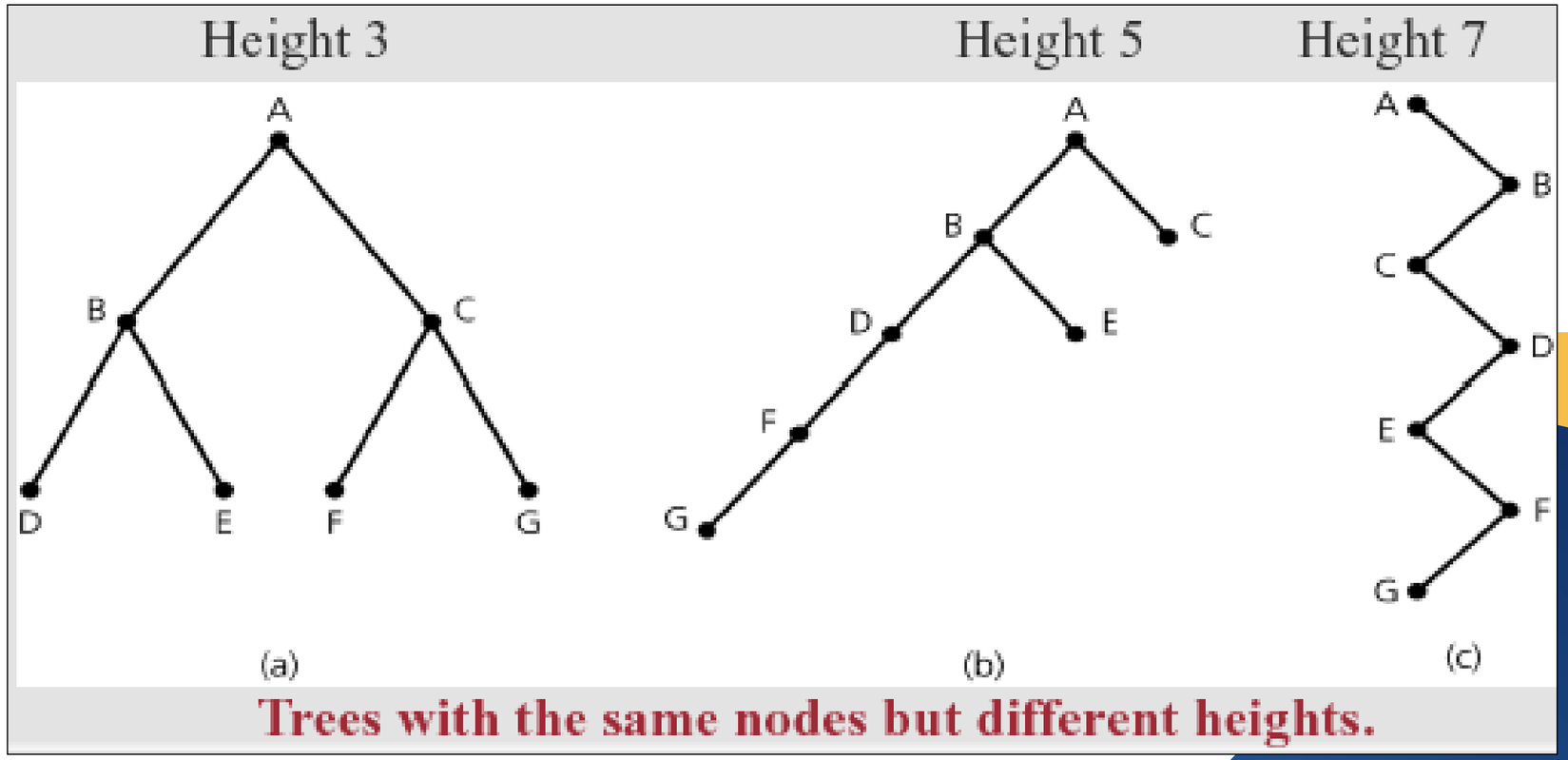
BINARY TREES

- A **binary search tree** is a binary tree but has the following properties for each node ***n***:
  - ***n***'s value is > all values in ***n***'s left subtree T_L
  - ***n***'s value is < all values in ***n***'s right subtree T_R
  - Both T_L and T_R are binary search trees.



BINARY TREES

- **Height of a tree** – number of nodes along the longest path from the root to a leaf.

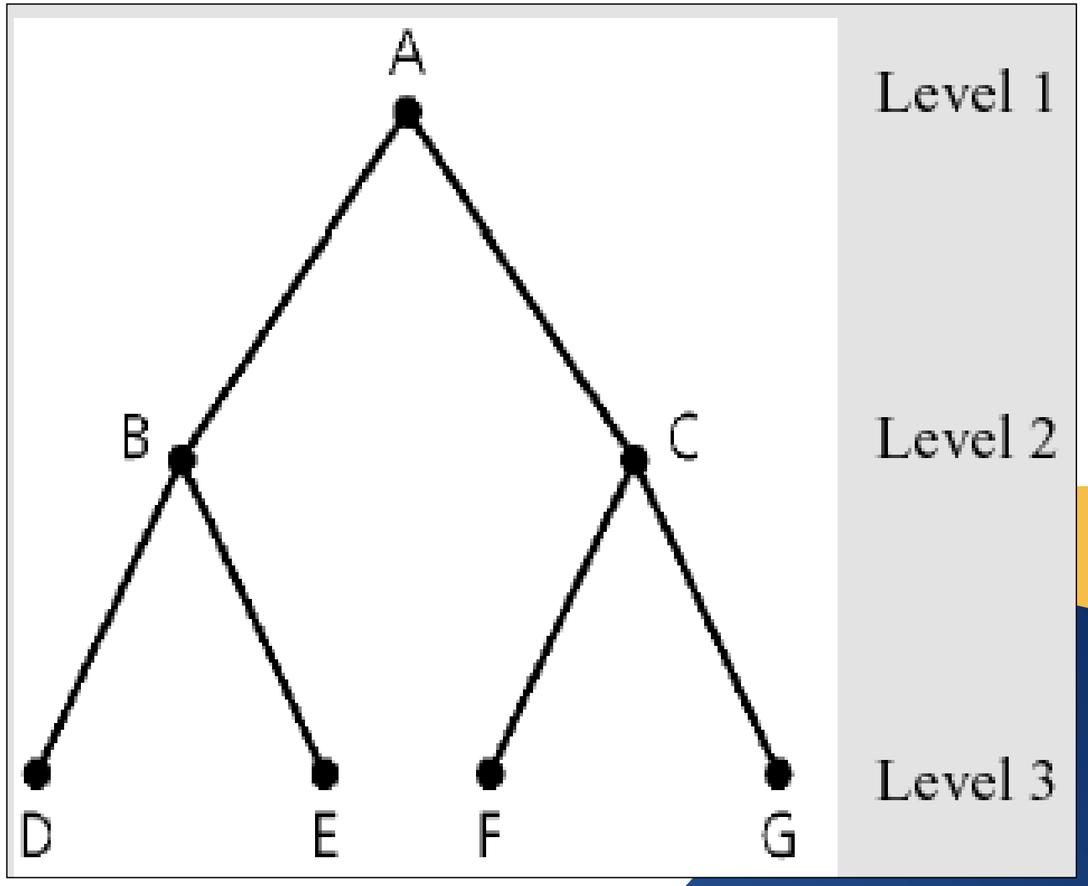


BINARY TREES

- **Level of a node** n in a tree, ***T***:
  - If ***n*** is the root of ***T***, it is at level 1
  - If ***n*** is not the root of ***T***,

    its level is 1 greater than the

    level of its parent



BINARY TREES

- Height of a tree, ***T*** defined in terms of the levels of its nodes..
  - If ***T*** is empty, its height is 0
  - If ***T*** is not empty, its height is equal to the maximum level of its nodes.
- A recursive definition of height
  - If ***T*** is empty, its height is 0
  - If ***T*** is not empty,

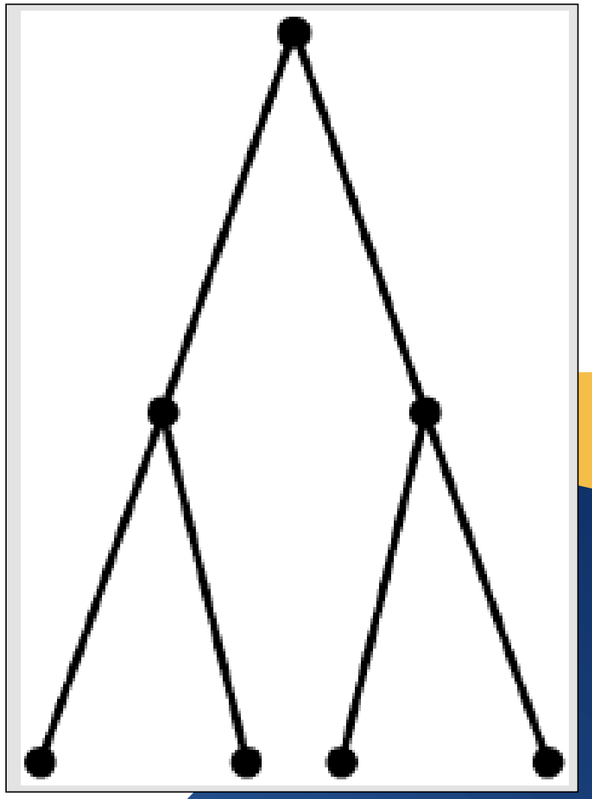
$$height(T) = 1 + \max\{height(T_L), height(T_R)\}$$



BINARY TREES

Full Binary Trees

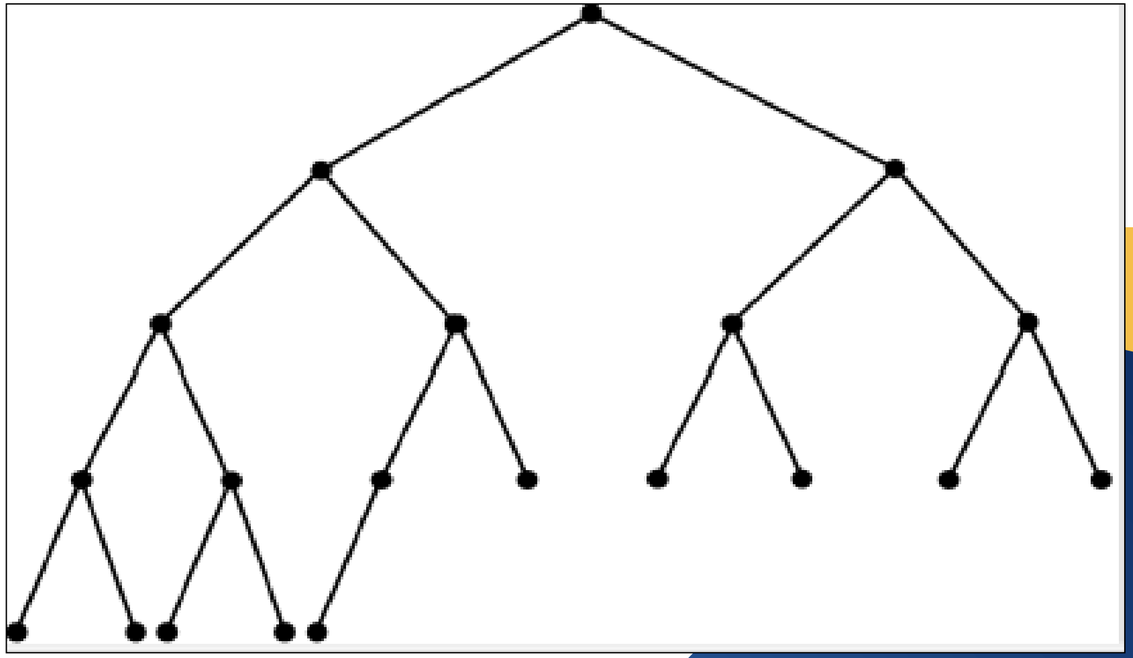
- A binary tree of height ***h*** is full if nodes at levels < ***h*** have two children each.
- Recursive definition:
  - If ***T*** is empty, ***T*** is a full binary tree of height 0
  - If ***T*** is not empty and has height ***h*** > 0, ***T*** is a full binary tree of root's subtrees are both full binary trees of height ***h – 1***.



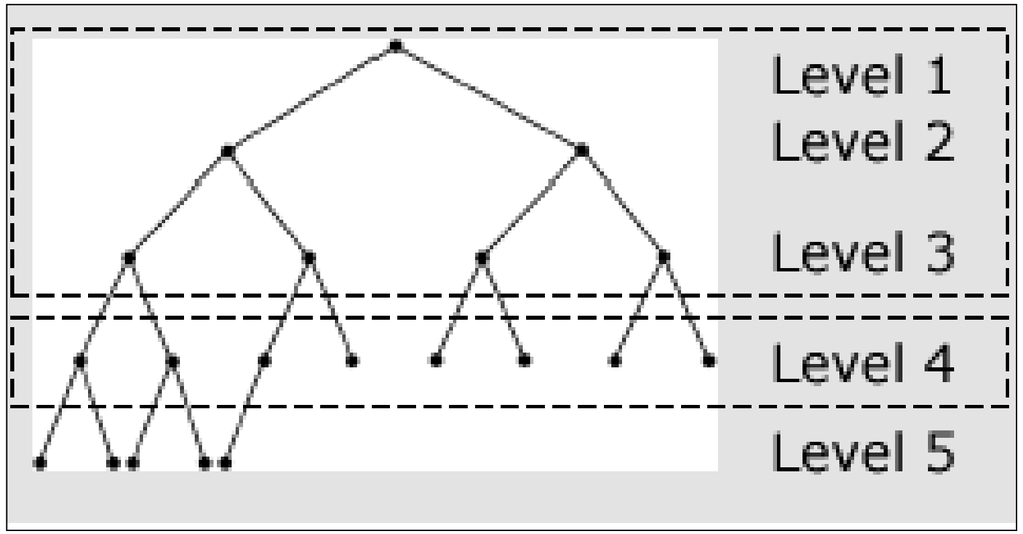
BINARY TREES

Complete Binary Trees

- A binary tree of height is **complete** if
  - It is full to level ***h – 1***
  - Level ***h*** is filled from left to right



BINARY TREES

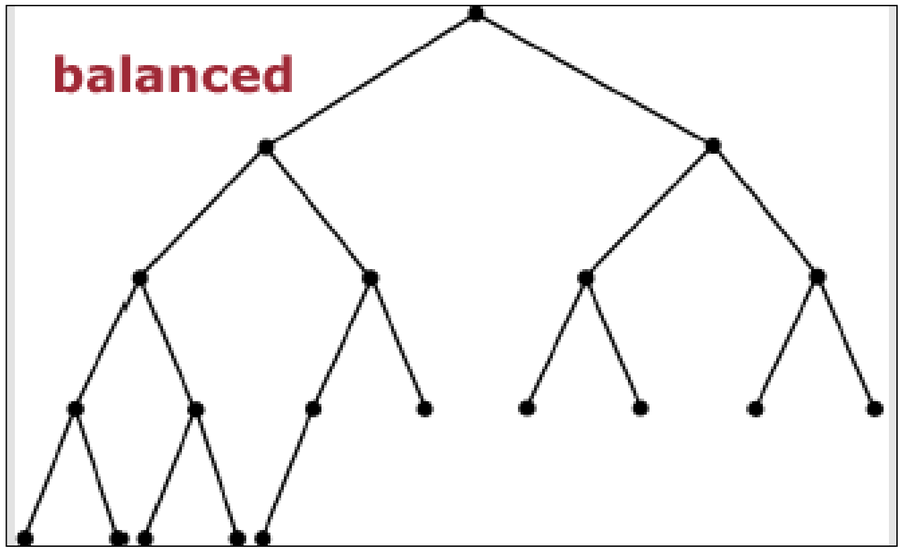


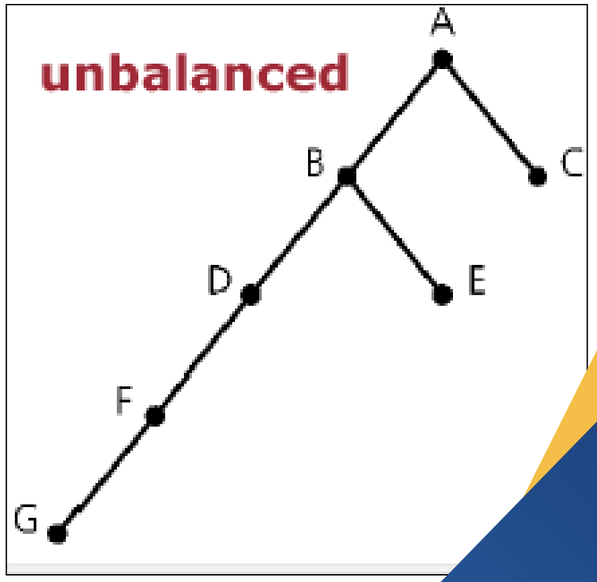
- Another definition:
  - A binary tree of height ***h*** is complete of:
    - All nodes at levels <= ***h-2*** have two children each, and
    - When a node at level ***h-1*** has children, all nodes to its left at the same level have two children each
    - When a node at level ***h-1*** has one child, it is a left child

BINARY TREES

Balanced Binary Trees

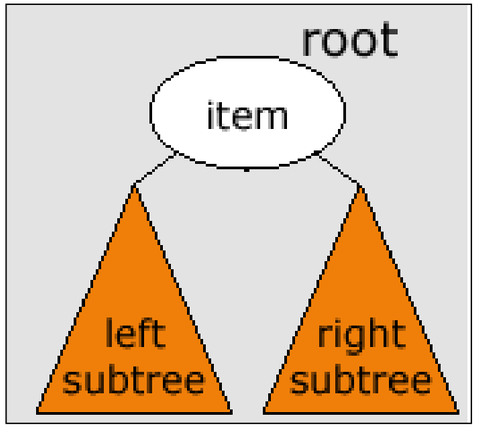
- A binary tree is balanced if the heights of any node's two subtrees differ by no more than 1
  - Complete binary trees are balances
  - Full binary trees are complete and balanced





ADT BINARY TREES

- Properties of data:
  - `TreeItemType` – the type of items stored in the binary tree. E.g., int
  - Root, left/right subtrees
- Operations:



- `createBinaryTree(in RootItem: TreeItemType) throw TreeException`

  Creates a **one-node** binary tree whose root contains `rootItem`.

  Throw `TreeException` if allocation fails.

ADT BINARY TREES

- `createBinaryTree(in rootItem:TreeItemType, inout leftTree:BinaryTree, inout rightTree:BinaryTree) throw TreeException`

  Create a binary tree whose root contains `rootItem` and has `leftTree` and `rightTree`, respectively, as its left and right subtrees.

  Makes `leftTree` and `rightTree` empty so they cannot be used to gain access to the new tree.

  Throw `TreeException` if allocation fails.

- `isEmpty():boolean {query}`

  Determines whether this binary tree is empty.

ADT BINARY TREES

- `getRootData():TreeItemType throw TreeException`

  Returns the data item in the root of a nonempty binary tree.

  Throws `TreeException` if the tree is empty.

- `setRootData(in newItem:TreeItemType) throw TreeException`

  Replace the data item in the root of this binary tree with `newItem`, if the tree is not empty.

  However, if the tree is empty, create a root node whose data item is `newItem` and inserts the new node into the tree.

  If a new node cannot be created, `TreeExcpetion` is thrown.

ADT BINARY TREES

- `attachLeft(in newItem:TreeItemType)`
  `throw TreeException`

  Attaches a left child containing `newItem` to the root of this binary tree.

  Throws TreeException if a new child node cannot be allocated.

  Also throws `TreeException` if the binary tree is empty or a left subtree already exists.

- `attachRight(in newItem:TreeItemType)`
  `throw TreeException`

ADT BINARY TREES

- `attachLeftSubtree(inout leftTree:BinaryTree)`
  `throw TreeException`

  Attaches leftTree as the left subtree of the root of this binary tree and makes `leftTree` empty so that it cannot be used to access this tree.

  Throws `TreeException` if the binary tree is empty or a left subtree already exists.

- `attachRightSubtree(inout rightTree:BinaryTree)`
  `throw TreeException`

ADT BINARY TREES

- `detachLeftSubtree(out leftTree:BinaryTree)`
  `throw TreeException`

  Detaches the left subtree of this binary tree's root and retains it in `leftTree`.

  Throw `TreeException` if the binary tree is empty.

- `detachRightSubtree(out rightTree:BinaryTree)`
  `throw TreeException`

ADT BINARY TREES

- `getLeftSubtree():BinaryTree`

  Returns a **copy** of the left subtree of this binary tree's root without detaching the subtree.

  Returns an empty tree if the binary tree is empty.

- `getRightSubtree():Binarytree`

ADT BINARY TREES

- `preorderTraverse(in visit:FunctionType)`

  Traverses this binary tree in **preorder** and calls the function `visit()` once for each node.

- `inorderTraverse(in visit:FunctionType)`

  Traverses this binary tree in **inorder** and calls the function `visit()` once for each node.

- `postorderTraverse(in visit:FunctionType)`

  Traverses this binary tree in **postorder** and calls the function `visit()` once for each node.

ADT BINARY TREES

- Building an ADT binary tree.

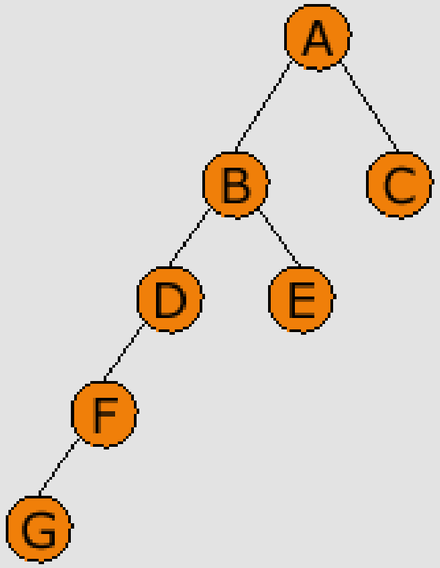
```
tree1.setRootData('F')
tree1.attachLeft('G')

tree2.setRootData('D')
tree2.attachLeftSubtree(tree1)

tree3.setRootData('B')
tree3.attachLeftSubtree(tree2)
tree3.attachRight('E')

tree4.setRootData('C')

binTree.createBinaryTree('A',tree3,tree4)
```



Initially, all trees exist but are empty.

APPLICATION

❖ **Traversals of a Binary Tree**

- A **traversal** visits **each node** in a tree.
  - You do something with or to the node during a visit.
  - For example, **display** or **modify** the data in the node.

- With the recursive definition of a binary tree, you can construct a recursive traversal algorithm.
  - If the tree is not empty, the traversal algorithm must perform three tasks:
    - Display the data in the root.
    - **Traverse** the two subtrees (recursive … smaller problems).
  - The base case: If the tree is empty, the algorithm takes no action.

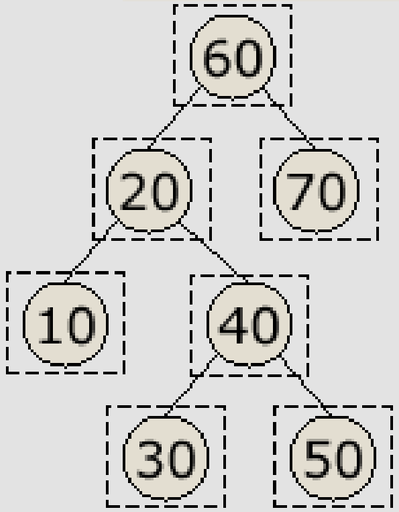
APPLICATION

- The algorithm has ***three choice of when to visit the root***.
  - **Preorder traversal**
    - Visit root before visiting its subtrees.
    - i.e., Before the recursive calls.
  - **Inorder traversal**
    - Visit root between visiting its subtrees.
    - i.e., Between the recursive calls.
  - **Postorder traversal**
    - Visit root after visiting its subtrees.
    - i.e., After the recursive calls.

APPLICATION

- The pseudocode of the preorder traversal algorithm:

```
Preorder (in binTree:Binarytree)
  if(bintree is not empty)  {
          Display the data in the root of binTree
          Preorder (Left subtree of binTree's root)
          Preorder (Right subtree of binTree's root)
  } // end if
```

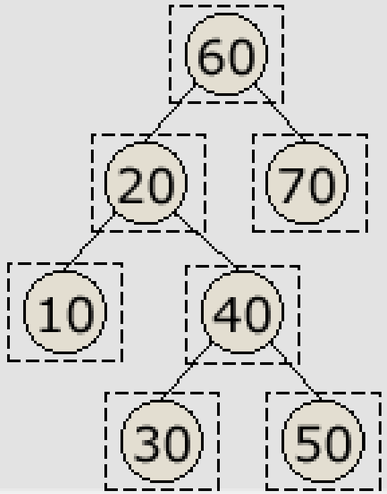


Preorder: 60 20 10 40 30 50 70

APPLICATION

- The pseudocode of the postorder traversal algorithm:

```
Postorder (in binTree:Binarytree)
  if(bintree is not empty)   {
          Postorder (Left subtree of binTree's root)
          Postorder (Right subtree of binTree's root)
          Display the data in the root of binTree
  } // end if
```



Postorder: 10 30 50 40 20 70 60

IMPLEMENTATION OF BINARY TREE

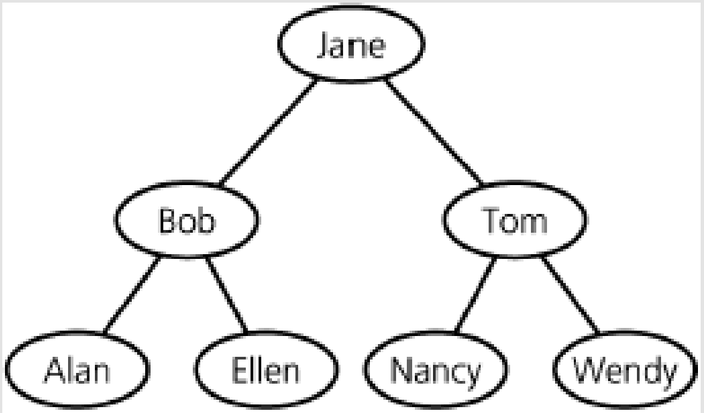
❖**Possible Representations of a Binary Tree**

- ADT binary tree implementation:
  - An **array-based** representation.
    - Uses an array of tree nodes.
  - A **pointer-based** representation.
    - Nodes have two pointers that link the nodes in the tree.

IMPLEMENTATION OF BINARY TREE

❖**The ADT Binary Search Tree**

- The ADT **binary search tree** is suitable for searching a tree for a particular data item.
- For each node n in a binary search tree:
  - n's value is greater than all values in its left subtree T_L.
  - n's value is less than all values in its right subtree T_R.
  - Both T_L and T_R are binary search trees.
- This organisation of data enables you to search a binary search tree for a particular data item efficiently.



IMPLEMENTATION OF BINARY TREE

- **Search key** (or simply **key**):
  - Is a field of a record that **uniquely** identifies it within a collection of records.
  - A data item in a binary search tree has a specially designated search key.
    - E.g., ID number of a person.

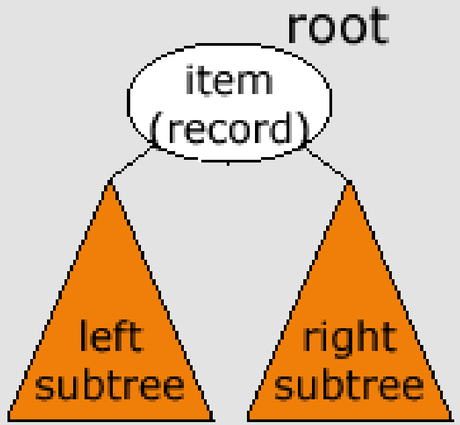
- KeyedItem class:
  - Contains the search key as a data field and a method for accessing the search key.
  - Prevents modification of the search-key value once an item is created.
    - Has only a constructor for initialising the search key

IMPLEMENTATION OF BINARY TREE

- A more specific recursive definition of a binary search tree:
  - For each node N, a binary search tree satisfies the following three properties:
    - N's search key is **greater** than all search keys in N's **left subtree** T_L.
    - N's search key is **less** than all search keys in N's **right subtree** T_R.
    - Both T_L and T_R are binary search trees.

IMPLEMENTATION OF BINARY TREE

- ADT design:
  - Properties of data:
    - `TreeItemType`: the type of items stored in the binary search tree. It should be based upon `KeyedItem`, which has a search key field of type `KeyType`.
    - Root, left/right subtrees.



IMPLEMENTATION OF BINARY TREE

- Operation contract:

  - `isEmpty():boolean {query}`

    determines whether this binary search tree is empty.

  - `searchTreeInsert(in newItem:TreeItemType) throw TreeException`

    Inserts `newItem` into this binary search tree whose items have distinct search keys that differ from newItem's search key.

    Throw `TreeException` if the insertion is not successful.

IMPLEMENTATION OF BINARY TREE

- `searchTreeDelete(in searchkey:KeyType) throw TreeException`

  Deletes from this binary search tree the item whose search key equals searchKey.

  If no such item exists, the operation fails and throws `TreeException`.

- `searchTreeRetrieve(in searchKey:KeyType, out treeItem:TreeItemType) {query} throw TreeException`

  Retrieves into `treeItem` the item in this binary search tree whose search key equals `searchKey`.

  If no such item exists, the operation fails and throws `TreeException`.

IMPLEMENTATION OF BINARY TREE

- `preorderTraverse(in visit:FunctionType)`

  Traverses this binary search tree in preorder and calls the function visit once for each item.

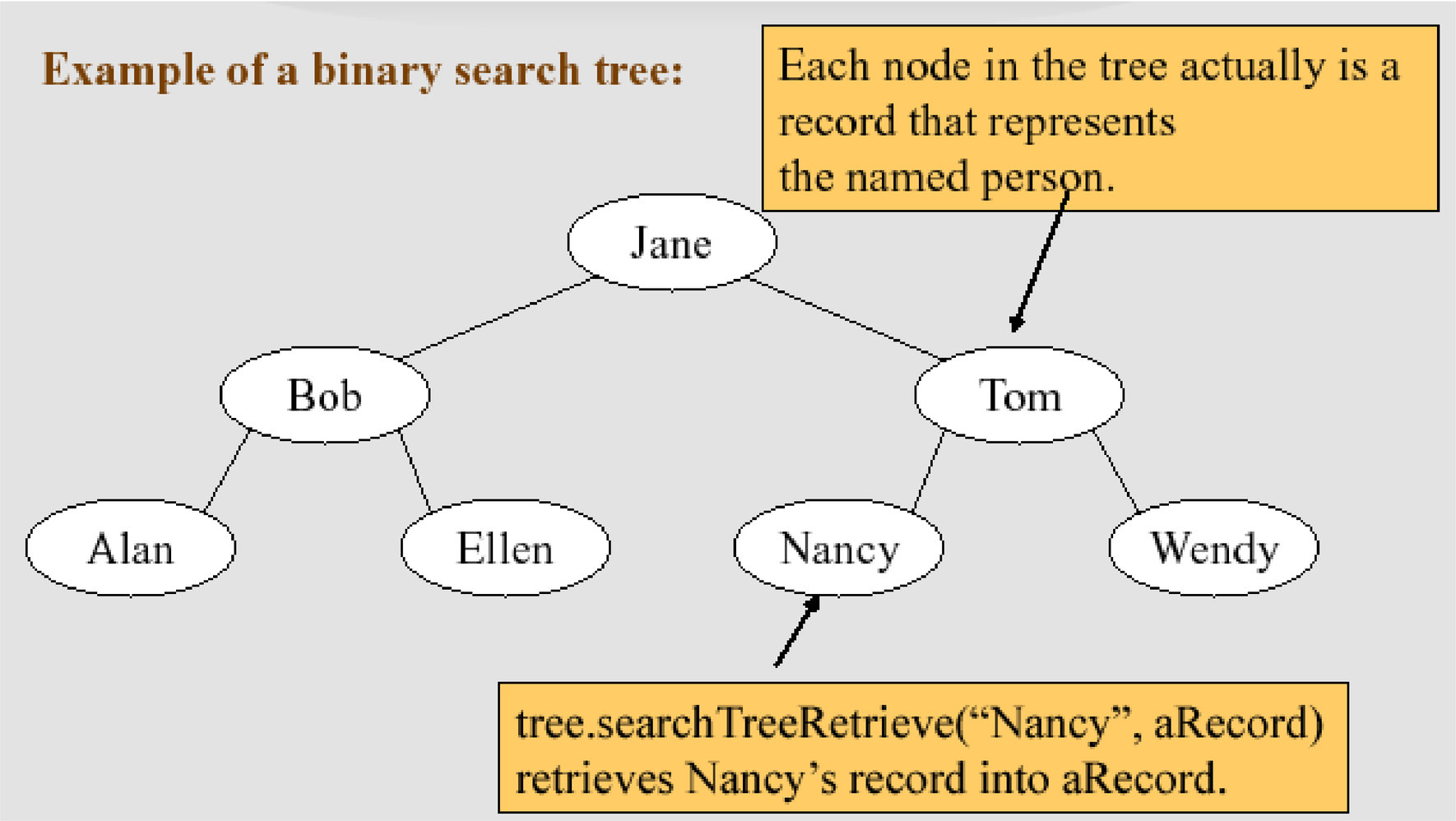
- `inorderTraverse(in visit:FunctionType)`

  Traverses this binary search tree in inorder and calls the function `visit` once for each item.

- `postorderTraverse(in visit:FunctionType)`

  Traverses this binary search tree in postorder and calls the function `visit` once for each item.

IMPLEMENTATION OF BINARY TREE



SUMMARY

- A **tree** is a hierarchical data structure that consists of **nodes** connected by **edges**, resembling an inverted tree structure with a **root node** at the top and various child nodes branching out beneath it.
- Each node contains a value or data and can have zero or more child nodes, making trees ideal for representing relationships and hierarchies, such as organizational structures, file systems, and family trees.

SUMMARY

- Trees also support various algorithms for traversing nodes, including **in-order, pre-order, and post-order traversal**, and are widely used in applications such as databases, artificial intelligence, and network routing.
- Understanding trees and their algorithms is essential for solving complex problems in computer science and software development.

SUMMARY

- Example of real-world applications:
  - File Systems (Directory Structure)
  - Game Development (Scene Graphs)
  - Website Navigation (DOM Tree)
  - Organizational Hierarchies
  - Library Book Classification
  - Tournament Brackets
  - Product Categories on E-commerce Sites